Code Explanation

Unit Test Explanation for Source Count Check Module

This document provides a step-by-step explanation of the unit tests for the source count check module. The tests are designed to verify the functionality of the module, which includes extracting source data, calculating the source record count, writing the count to a target file, and running the full source count check process.

Overview of the Code
The code is a set of unit tests written in Python using the unittest framework. It tests the functionality of the source count check module, which is responsible for extracting data from a source, calculating the record count, and writing the count to a target file. The tests cover various scenarios, including successful extraction, calculation, and writing, as well as handling overflow cases.

Step 1: Setting Up the Test Environment
The test class `TestSourceCountCheck` sets up a Spark session and creates sample test data. This step is necessary to provide a test environment for the subsequent tests.

```
@classmethod
def setUpClass(cls):
    cls.spark = (SparkSession.builder
                 .appName("TestSourceCountCheck")
                 .master("local[*]")
                 .getOrCreate())

    data = [
        (1, 101, 201, 301, "POL123", "ACTIVE"),
        (2, 102, 202, 302, "POL124", "PENDING"),
        (3, 103, 203, 303, "POL125", "ACTIVE"),
        (4, 104, 204, 304, "POL126", None),
        (5, 105, 205, 305, "POL127", "CLOSED")
    ]

    columns = [
        "T_TX_REQUEST_REQUEST_ID",
        "T_TX_BASIC_TRANS_ID",
        "T_TX_RELATION_TRANS_REL_ID",
        "T_TX_REQ_POL_REQUEST_POLICY_ID",
        "STG_POLICY_ID",
        "T_TX_REQUEST_REQUEST_STATUS"
    ]

    cls.test_df = cls.spark.createDataFrame(data, columns)
```

Step 2: Testing the Extraction of Source Data
The `test_extract_source_data` method tests the extraction of source data using SQL. It sets up a mock Spark session and verifies that the correct options are set for the extraction.

```
@patch("src.source_count_check.SparkSession")
def test_extract_source_data(self, mock_spark):
    mock_spark_instance = MagicMock()
    mock_read = MagicMock()
```

```
    mock_spark_instance.read.format.return_value = mock_read
    mock_read.option.return_value = mock_read
    mock_read.load.return_value = self.test_df

    test_sql = "SELECT * FROM ZSYSE2EDEV.STG_E2E_AC_TXDBH_DATA WHERE
 T_TX_REQUEST_REQUEST_STATUS = 'ACTIVE'"
    result_df = extract_source_data(mock_spark_instance, test_sql)

    mock_spark_instance.read.format.assert_called_with("jdbc")
    mock_read.option.assert_any_call("dbtable", f"({test_sql})")

    self.assertEqual(result_df, self.test_df)
```

Step 3: Testing the Calculation of Source Record Count
The `test_calculate_source_count` method tests the calculation of the source record count. It executes the `calculate_source_count` function and verifies that the result is correct.

```
def test_calculate_source_count(self):
    result_df = calculate_source_count(self.test_df)

    result_list = result_df.collect()

    self.assertEqual(len(result_list), 1)
    self.assertEqual(result_list[0]["SOURCE_RECT"], "5")
```

Step 4: Testing the Handling of Overflow in Source Count
The `test_calculate_source_count_overflow` method tests the handling of overflow in the source count. It creates a mock dataframe with a large number of rows and verifies that the function returns "OVERFLOW" in such cases.

```
def test_calculate_source_count_overflow(self):
    large_count_df = self.spark.createDataFrame(
        [(i,) for i in range(1_234_567_890)],
        ["id"]
    )

    large_count_df = large_count_df.select(
        F.lit(1_234_567_890).alias("SOURCE_REC_COUNT")
    )

    with patch("pyspark.sql.DataFrame.agg") as mock_agg:
        mock_agg.return_value = large_count_df
        result_df = calculate_source_count(self.test_df)

        result = result_df.collect()[0]["SOURCE_RECT"]
        self.assertEqual(result, "OVERFLOW")
```

Step 5: Testing the Writing of Count to Target File
The `test_write_to_target` method tests the writing of the count to a target file. It creates a simple count dataframe and verifies that the function completes without errors.

```
def test_write_to_target(self):
    count_df = self.spark.createDataFrame([("5",)], ["SOURCE_RECT"])

    with tempfile.TemporaryDirectory() as temp_dir:
        output_path = temp_dir
        output_filename = "test_count.csv"

        with patch("os.path.join", return_value=f"{output_path}/{out
put_filename}"):
            with patch("src.source_count_check.SparkSession.getActiv
eSession") as mock_session:
                mock_session.return_value = self.spark

                write_to_target(count_df, output_path, output_filena
me)
```

Step 6: Testing the Full Source Count Check Process

The `test_run_source_count_check` method tests the full source count check process. It sets up mocks for the various functions involved and verifies that they are called with the correct parameters.

```
@patch("src.source_count_check.extract_source_data")
@patch("src.source_count_check.calculate_source_count")
@patch("src.source_count_check.write_to_target")
@patch("src.source_count_check.create_spark_session")
def test_run_source_count_check(self, mock_create_spark, mock_write,
 mock_calculate, mock_extract):
    mock_spark = MagicMock()
    mock_create_spark.return_value = mock_spark

    mock_source_df = MagicMock()
    mock_extract.return_value = mock_source_df

    mock_count_df = MagicMock()
    mock_calculate.return_value = mock_count_df

    test_sql = "SELECT * FROM ZSYSE2EDEV.STG_E2E_AC_TXDBH_DATA"
    test_output_path = "/tmp/test_output"
    test_output_filename = "test_count.csv"

    run_source_count_check(test_sql, test_output_path, test_output_f
ilename)

    mock_create_spark.assert_called_once()
    mock_extract.assert_called_once_with(mock_spark, test_sql)
    mock_calculate.assert_called_once_with(mock_source_df)
    mock_write.assert_called_once_with(mock_count_df, test_output_pa
th, test_output_filename)
```